

FoggyTrust: Robust Federated Learning with Hierarchical Trust Networks

1st Emmanuel Rassou

School of Engineering and Applied Sciences (SEAS)

Harvard

Cambridge, MA, USA

emmanuel_rassou@college.harvard.edu

2nd Tomas Gonzalez

School of Engineering and Applied Sciences (SEAS)

Harvard

Cambridge, MA, USA

tsgonzalez@college.harvard.edu

Abstract—Byzantine-robust federated learning seeks to protect distributed model training from malicious or corrupted clients without requiring access to their private data. FLTrust addresses this challenge by introducing a trusted server-side root dataset that assigns trust scores to client updates for more robust aggregation. In this work, we propose FOGGYTRUST, a hierarchical extension of FLTrust that localizes trust computation to fog nodes, allowing the framework to better handle globally heterogeneous data while preserving robustness within locally homogeneous client groups. We further show that this two-level architecture can simultaneously address distribution mismatch in trust estimation and client drift across groups by combining local trust-based aggregation with heterogeneity-aware global optimizers such as FedAdam and SCAFFOLD. Across benchmark datasets, FOGGYTRUST achieves its strongest gains on more challenging heterogeneous settings, particularly on CIFAR-10 under Krum and Trim attacks, where it achieves an over 50% improvement over FLTrust. We also test FOGGYTRUST in a real-world safari dataset to show the promise of hierarchical trust networks for robust federated learning in socially impactful, safety-critical settings such as distributed wildlife monitoring.

Index Terms—Federated Learning, Fog Computing, Model Safety

I. INTRODUCTION

Federated Learning (FL) is an emerging distributed machine learning paradigm which enables learning from clients without exposing their data [1]–[4]. Specifically, FL uses a centralized service provider which maintains the global model. At each iteration of the learning algorithm, the service provider (i.e. Google, Apple, etc) sends the global model to each client (i.e. smartphones, etc.), which then uses on-device data to make local updates to the model. The local updates are then sent back to the service provider, which uses an aggregation rule to fold all local updates into one global update.

Since each client only sends their update to the parameters, it does not have to expose the data to the service provider, promising valuable advantages in privacy with various use cases [5], [6]. However, the privacy is a double edged sword. Without access to the data used to make the local update, the service provider is unable to validate whether updates are correct, uncorrupted, and non-malicious. Thus, FL setups are vulnerable to byzantine attacks, where malicious clients may poison their local data [7], [8] or local updates [9]–[12], worsening the performance of the global model.

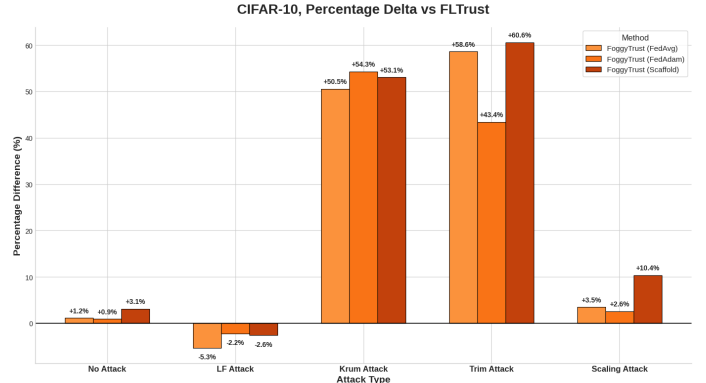


Fig. 1. Comparing our FoggyTrust algorithms as a percentage of the FLTrust(SOTA baseline) scores on the CIFAR-10 dataset across all attacks. Our method FoggyTrust performs the best for harer attacks like Krum and Trim attack, but has limitations for data poisoning attacks.

This issue is particularly pertinent to multi-agent coordination. For example, a multi-agent system may rely on a vision model trained by FL to identify aspects of their environment. If malicious agents harm the model, the agents will struggle to identify relevant aspects of their environment and accomplish their task.

II. PREVIOUS WORK

A. Background on Byzantine-Robust FL Methods

Byzantine-robust FL methods seek to solve that problem. That is, they attempt to enable FL to successfully occur in situations where clients may be malicious or corrupted. Many of these methods, including Krum [13], Trimmed mean [14], and Median [14], are robust aggregation rules that use statistical techniques to determine if an update is malicious. However, studies have shown that these aggregation rules are still vulnerable to carefully crafted attacks [9]. These attacks exploit the statistical techniques in a variety of ways.

Fang et al. [9] created a general framework for these attacks, which adapts to each robust aggregation method. In essence, their framework creates an optimization problem where they attempt to update the global model along the direction most opposite to a non-malicious update. They tested their framework on the Krum and Trimmed mean aggregation

methods, and showed that their attacks, the Krum Attack and Trim Attack respectively, severely degraded performance.

B. FLTrust Algorithm

Cao et al. [3] recognized that robust-aggregation methods relying on statistical techniques alone would remain vulnerable to carefully crafted attacks. As a result, they extend previous methods by allowing the service provider to bootstrap trust. By developing a notion of how much the service provider trusts each local update, the service provider can accept trustworthy updates and reject or minimize untrustworthy ones.

They provide this notion of trust by crafting a small (< 100 samples) root dataset for the service provider, which is unique for FL where data is typically entirely decentralized. The root dataset must be non-malicious and representative of the data seen in the clients.

The first two steps of an iteration of the algorithm are largely unchanged from typical FL methods. First, they synchronize the models by sending the service provider’s global model, w to the clients participating in this round of updates. Secondly, the clients make local updates g_i to the model by fine tuning on the data that is present on their devices. Novelly, they also compute the service provider’s local update, g , based on the carefully crafted root dataset.

Next, each clients local update g_i is compared to g by taking the cosine similarity of the two updates. A more aligned update (cosine similarity closer to 1) is considered more trustworthy, whereas less aligned updates (cosine similarity closer to -1) are considered less trustworthy. By the construction of their aggregation rule, a local update with a negative trust score would still contribute toward the global model, despite them being considered very untrustworthy. Thus, they ReLU clip the trust score to ensure updates with negative cosine similarities aren’t included. Finally, they normalize each local update g_i to match the magnitude of g , protecting against scaling attacks, where malicious agents attempt to have a disproportionately large effect on the global model [11]. Then, they aggregate the normalized local updates by weighing each update g_i proportionally to its ReLU-clipped trust score TS_i .

Their defense model successfully defended against the Krum and Trim attack, among others, with some experiments showing no reduction in accuracy in some instances, and only a small reduction in others [3]. However, their defense relies a strong assumption that weakens its practicality: FLTrust assumes that the data each client possesses is relatively independent and identically distributed (iid).

In reality, the applications that employ FL often have very heterogeneous information. For example, consider the next-word application of FL on Android GBoard [5]. This is trained on language data from billions of different users, which certainly don’t speak in the same manner.

Heterogeneity presents a problem for FLTrust for two reasons. First, the heterogeneity of a dataset can cause its root of trust to not be sufficiently representative of all clients. If we have an unrepresentative root dataset, then our trust scores will be brittle. Since the aggregation rule directly depends on

our trust scores, it begins to break down the algorithm. For example, FLTrust may deem updates untrustworthy even if they’re not malicious, as they sufficiently differ from the root datasets update due to the non-iid-ness of the data.

Secondly, non-iidness in data in FL is a problem in and of itself. Reddie et al. describes two primary issues FedAvg runs into in certain applications [15]. First, it can suffer from “client drift,” where the local updates begin to deviate from the global model as they move toward locally optimal solutions. Secondly, the method is non-adaptive, making it unable to adapt to client variability.

Several methods have been created to correct for these circumstances. To address the first issue, SCAFFOLD [4] introduces client and server control variates that act as correction terms during local training, reducing the bias caused by heterogeneous client data and mitigating client drift. By keeping local updates better aligned with the global objective, SCAFFOLD improves convergence in non-IID settings. To address the second issue, FedAdam [15] replaces FedAvg’s standard server update with an adaptive optimization method, enabling the server to adjust more effectively to variability in client updates across rounds. These two approaches therefore target complementary weaknesses of FedAvg under heterogeneity: SCAFFOLD reduces drift in local optimization, while FedAdam improves the adaptivity of global aggregation. However, to our knowledge, no heterogeneity-correcting methods that are byzantine-robust exist, and FLTrust makes no effort to resolve these heterogeneity problems within their techniques.

Thus, global non-iidness presents a substantial issue for FLTrust, but it is extremely prevalent in application. However, we believe that a weaker assumption may be more realistic: that data can live in families of locally homogeneous pockets despite a globally heterogeneous structure. For instance, consider the GBoard example, and assume that it serves all English speakers in the United States. Then, people in Wisconsin may talk very similarly to each other, but differently than people in Alabama. Our primary objective is to exploit *local homogeneity* to outperform FLTrust in highly heterogeneous environments.

Specifically, in this paper, we have three main contributions over the baseline method FLTrust:

- We employ root datasets across many fog nodes instead of one at the global server, localizing the trust scores to groups of clients that are more homogeneous, and aggregate the updates at that level with the locally computed trust scores.
- We create a hierarchical nature to fold in updates computed at each fog node, and swap out the 2nd-level aggregator with other federated learning aggregation rules that are more robust to client drift.
- We test our method against FLTrust in a real-world dataset such as SnapshotSafari [16].

Our code is publicly accessible at the following link: <https://github.com/emmanuel2406/FoggyTrust.git>

III. FOGGYTRUST

We propose FOGGYTRUST, a hierarchical FL method that bootstraps trust and performs well in globally heterogeneous applications that possess local homogeneity. See Figure 2 for a visual overview.

FoggyTrust extends FLTrust in the following manner. Rather than having a single global server run the FLTrust algorithm for all clients, the service provider instead communicates with fog nodes. The fog nodes serve as local service providers for data that we assume to be relatively iid. Following our previous example, there could be a fog node at each state, where people are assumed to talk relatively similarly. We construct a root dataset for each of these fog nodes, under the assumption that it is relatively representative of its clients data.

Thus, at each iteration of the algorithm, clients make local updates which they send to their respective fog node rather than the global server. Each fog node aggregates the client data corresponding using FLTrust to have a fog node level local update. Then, each of the fog nodes local updates send their aggregated update to the global server. The global server then uses an aggregation rule (such as FedAvg [2]) to fold in the updates of each fog node.

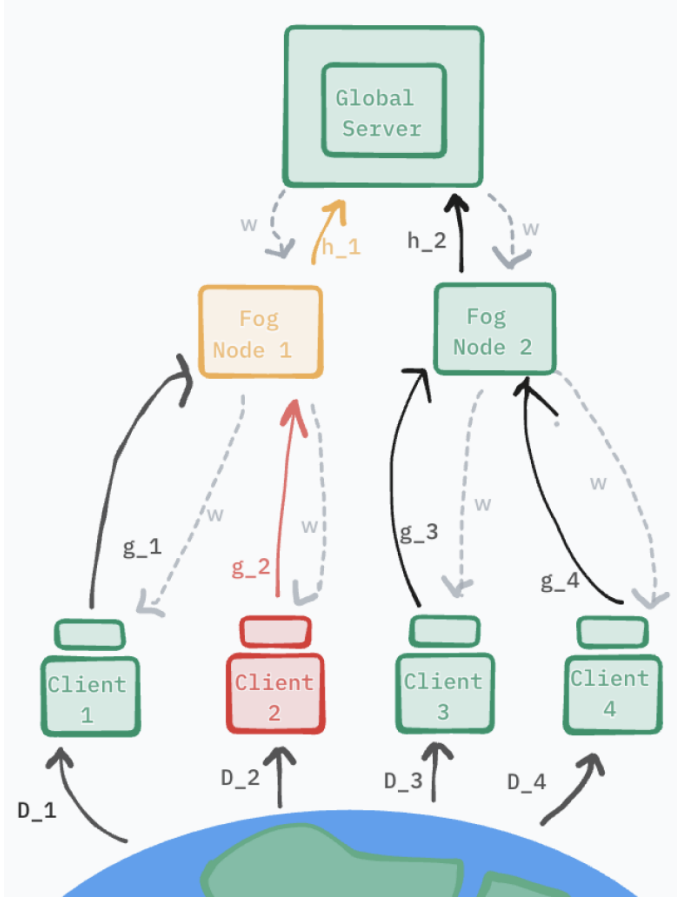


Fig. 2. High level overview of FoggyTrust with FedAvg, where **stage i** applies FLTrust between the clients and the fog nodes and **stage ii** applies FedAvg between the weight updates of each fog node and the global server.

FoggyTrust with FedAvg overcomes the heterogeneity between server updates and each client update. Since each local group is assumed to be relatively iid, the root dataset at each fog node can thus be representative of the data within. This ensures that the trust scores are accurate, and the aggregation rule performs as intended at the lower level. The higher level aggregation does not compute trust scores, thus all FLTrust-level issues with heterogeneity are addressed by this approach.

We also seek to address the second problem of heterogeneity, which in our instance would be client drift between fog nodes. Despite each fog node being relatively iid within itself, the local regions are still globally heterogeneous. Thus, "client drift" is still a potential issue. However, our hierarchical nature enables us to resolve this using existing aggregators specialized for heterogeneity: SCAFFOLD [4] mitigates client drift through control variates that correct biased local updates, whereas FedAdam [15] uses adaptive server-side optimization to improve convergence and accuracy in heterogeneous environments. Accordingly, we modify our second-level aggregation mechanism to better account for heterogeneity and enhance performance.

To our knowledge, FoggyTrust is the first byzantine-robust aggregation method specialized for heterogeneous applications.

IV. EVALUATION

A. Experimental Setup

We aim to test the robustness and generality of three new learning methods:

- 1) FOGGYTRUST(FEDAVG): Our vanilla FoggyTrust algorithm with FedAvg [2] used to aggregate at the second level
- 2) FOGGYTRUST(FEDADAM): FedAdam is used to aggregate at the second level [15]
- 3) FOGGYTRUST(SCAFFOLD): SCAFFOLD is used to aggregate at the second level [4]

The exact parameters for each dataset is given in Table I.

1) **Datasets:** We test our FoggyTrust on four image classification datasets of varying difficulty: MNIST, Fashion-MNIST, CIFAR-10 and Snapshot Safari. Our first step was to replicate the results from FLTrust [3] by building upon the official implementation by one of the main authors <https://minghongfang.com>.

We follow previous work [3] to partition the dataset across different clients. For MNIST, FashionMNIST and CIFAR-10, each with 10 label classes, we assign each client a biased label so that 50% of their data points are from that label and the rest are uniformly distributed across the other label classes, so the other groups have probability $\frac{1-0.5}{10-1} \approx 0.056$. This simulates non-IID local training data.

MNIST: MNIST [17] consists of 10 digits from 0 to 9 with 60,000 training examples

Fashion-MNIST: Fashion-MNIST [18] has 60,000 fashion images distributed across 10 categories of different items in the fashion domain

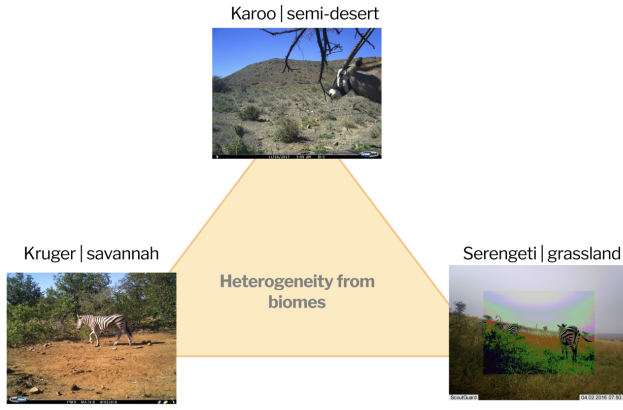


Fig. 3. Overview of Snapshot Safari setup where we choose three tapped camera environments that are highly heterogenous with respect to each other

CIFAR-10: CIFAR-10 [19] has RGB color images with 50,000 training examples distributed across 10 semantic categories.

Snapshot Safari: Snapshot Safari [16] is a real-world dataset consisting of 4 Million images from 15 camera tapping projects in the safari. We pick three of these projects which have different biomes, as seen in Figure 3. Namely, Karoo in South Africa being a semi-desert, Kruger in South Africa being a savanna, and Serengeti in Tanzania being a grassland. The task here is a multi-label classification task with 151 categories, but without a baseline to ensure proper replication. With limited compute available on this more expansive dataset, we only train for 200 iterations per configuration.

2) *Global Models:* To show the generality of our method we train different types of models across the different datasets. Particularly, we use a Convolutional Neural Network (CNN) identical to the one used in [3] for MNIST and Fashion-MNIST. CIFAR-10 and Snapshot Safari are significantly harder tasks so we train a model based on the ResNet-20 architecture [20].

3) *Poisoning Attacks:* Each attack uses the same parameters as in [3]. Label flipping (LF) attack is our only data poisoning attack, which essentially flips l to be $M - l - 1$ for a dataset with M classes.

We also test on Trim, Krum, and Scaling Attack which are local model poisoning attacks. Trim and Krum are both optimized against the Trimmed-Mean and Krum aggregation rules. The Scaling Attack is a targeted attack that scales its local update contribution by $\lambda \gg 1$ before the server handles it. We also measure the attack success rate (lower is better) on top of the usual test accuracy for this attack.

4) *Extension Setup:* For MNIST, Fashion-MNIST and CIFAR-10, we have 100 clients each and so we group them into client clusters based on their biased label. This gives 10 groups of 10 clients. Each group will be assigned a “fog node” where a local FLTrust algorithm will perform aggregation and then our FoggyTrust hierarchical setup allows an aggregation of each fog node’s update vector to the global server.

Analogous to the sampling of the reference dataset for the global server in FLTrust [3], we sample each fog node’s reference dataset based on the same biased probability used for their client groups.

For Snapshot Safari, we have 3 groups of 10 clients, one for each biome. In this case, the heterogeneity is implicit in the differences in camera tap locations. The fog node reference datasets are similarly sampled.

B. Results

1) *MNIST:* As seen in Table II, we find FoggyTrust(FedAvg) beats all the other baselines (including FLTrust) under no attack, Trim and Scaling Attack. However, FLTrust still performs best or equal best for the Label Flipping and Krum Attack. FLTrust also has the best attack success rate for the scaling attack.

The second level aggregator (both FedAdam and SCAFFOLD) does not seem to help final test accuracies. These methods aim to help the second problem addressed which is the client drift between different client groups. The distribution of digit classes may be close enough under MNIST that the overhead of a more complicated aggregation rule outweighs the potential benefits of mitigating this issue.

2) *Fashion-MNIST:* We compare our vanilla FoggyTrust with each of the previous defense methods in Table III. FoggyTrust(FedAvg) beats all the other baselines under Krum and Scaling attack. FLTrust performs quite well under this dataset beating our algorithm under no attack and Trim Attack.

The data poisoning attack, label flipping attack has the simplest baseline FedAvg perform the best: 0.8943 vs 0.8813 for FLTrust & 0.8784 for FoggyTrust(FedAvg).

The margins between the methods are still small enough to partially be attributed to noise in training, and so we also consider harder datasets to see a more clearer separation.

3) *CIFAR-10:* We compare all three of our methods to FLTrust in Table IV. Under all attack setups except the Label Flipping attack, all three of our new methods beat FLTrust.

Comparing the different versions of FoggyTrust, the harder task provided by the CIFAR-10 dataset shows promising gains by swapping out the FedAvg algorithm at the second level of our hierarchical algorithm. FoggyTrust with FedAdam performs the best under with a score of 0.5501. FoggyTrust with SCAFFOLD gives more consistent gains with the best scores under no attack, Trim Attack and Scaling attack (both attack success rate and testing accuracy). This method also beats FoggyTrust with FedAvg under Krum attack.

Figure 1 highlights the magnitude of difference between the method’s performance. By measuring it as a percentage delta of the FLTrust baseline, we see that FLTrust does not perform as well for the Krum and Trim attacks which both target the direction of the gradient update vectors. On the contrary, FoggyTrust calculates the trust scores at only a local lever which preserves the robustness of the model’s federated learning especially under local model poisoning attacks.

As a disclaimer, our experiments do not replicate the abnormally high accuracy results from [3] under this dataset. We rationalize the difference from other settings to be due to the training of a larger model Resnet-20 which may have had an undisclosed initialization or checkpoint, or the dataset may have had some preprocessing step as a necessary transformation.

4) *Snapshot Safari*: The full results are shown in Figure 4. We only test on No attack, Label Flipping attack and Trim Attack. The simplest algorithm, FedAvg, performs the best under all three settings. For the two attack settings, this seems counter-intuitive as, this method should be the most susceptible to attacks.

Our FoggyTrust algorithm does offer some value by bridging the gap in performance between FedAvg and FLTrust. This is especially true for Trim Attack, where FLTrust has a drop in their test accuracy from 0.6383 to 0.4297, while FoggyTrust has a more graceful drop from 0.6455 to 0.6029.

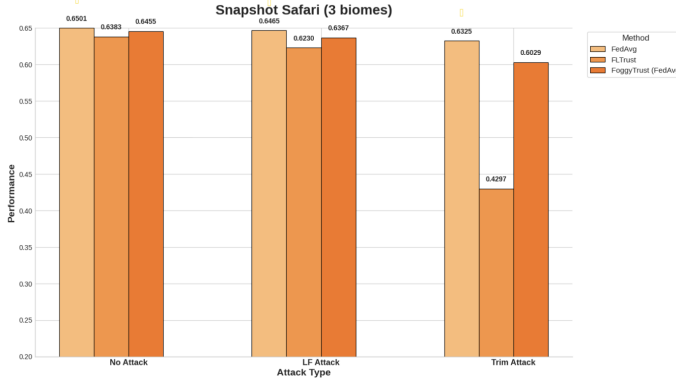


Fig. 4. Comparing our FoggyTrust algorithms as a percentage of the FLTrust scores on our subset of the Snapshot Safari dataset across 3 attack settings

C. Partitioned FoggyTrust

By default, we replicate the size of the reference datasets given to each client group. We also tested a partitioned version of our FoggyTrust algorithm, where we instead *partition* the server root dataset equally across all client groups. So for the datasets with 10 fog nodes, each one will have a reference dataset of 10 data points, instead of 100. This leads to slower convergence compared to our original version with accuracy scores of 0.7882 (vs 0.9627) under no attack and 0.7840 (vs 0.9604) under scaling attack for MNIST. [3] has tuned the dataset size to be 100, and so we adopt this size length even though it would mean a higher overhead for the model administrator to collect data as we scale the number of client groups.

V. DISCUSSION AND LIMITATIONS

A. FoggyTrust demonstrates improved robustness under stronger attacks and more challenging datasets

The most significant performance gains of FoggyTrust over the FLTrust baseline are observed on the CIFAR-10 dataset under both Trim and Krum attacks. Notably, these attacks

are specifically designed to manipulate the direction of client update vectors. In scenarios where the server-side reference dataset is not representative of the heterogeneous client distributions, FLTrust may assign low trust scores to benign client updates while inadvertently allowing adversarial updates to influence the global aggregation.

In contrast, the hierarchical structure of FoggyTrust yields more reliable trust estimation by leveraging local homogeneity within each fog node. By computing trust scores at the level of relatively homogeneous client groups, the method mitigates the mismatch between the reference dataset and client distributions that can degrade FLTrust’s effectiveness.

Although the grouping strategy in synthetic settings such as CIFAR-10 explicitly enforces homogeneity, similar structure often emerges in real-world federated learning deployments. In particular, client data distributions are frequently correlated with latent factors such as geographic location, leading to naturally clusterable subpopulations. This observation suggests that the assumptions underlying FoggyTrust are not only practical but also aligned with real-world data heterogeneity patterns.

B. Modular Advantage of Two-Level Aggregation

One of the benefits that comes with having two levels of aggregations is that we can swap in any aggregation rule between the fog node and the global server level. While FedAvg should do well, we note that it suffers from client drift and unadaptability, which will be exacerbated at the second level where client groups are heterogeneous with respect to each other.

From the CIFAR-10 experiment, we see that SCAFFOLD [4] consistently had gains over our vanilla implementation of FedAvg at the second level. SCAFFOLD introduces control variates specifically to correct the client drift. For Krum, Trim and Scaling attacks, this makes updates less sensitive to local deviations by aligning each update specifically with a global objective. This reduction in variance across fog node groups allows for a more robust learning process that makes it tougher for malicious clients to skew the gradient direction of the global model.

SCAFFOLD is just one possible aggregation rule. We also tested FedAdam [15] which focused more on the global aggregation rule than the local optimization and so it was not as robust for all attack vectors. Our paper hopes to open up the possibilities for future trust network architectures which potentially synergize better with the trust score calculations on level one.

C. Limitations for Data Poisoning Attacks

Although our hierarchical extension proves to be valuable for local model poisoning attacks, the gains do not translate over to data poisoning attacks. Specifically, the label flipping attack for the Fashion-MNIST and CIFAR-10 datasets.

The main reason for this shortfall is due to the silent nature of the label-flipping attacks. The resulting gradients for training on the wrong labels are still mathematically plausible

and consistent with one another. For trust score calculation, if more of the malicious clients updates are higher weighted than the honest updates, this will degrade the accuracy of the model being trained.

Because the trust scores are calculated for 10 clients in the case of FoggyTrust (within every client group) compared to 100 in the case of FLTrust, the room for error is a lot smaller. This is coupled with the notion that updates will have higher variance for more complicated data distributions such as those sampled from Fashion-MNIST and CIFAR-10. Future work should investigate the scaling laws as we increase the number of clients to determine the optimal ratio of clients per fog node.

D. Deployment in the Wild

While the majority of our experiments are conducted on synthetic benchmarks, we additionally evaluate the practicality of FoggyTrust on a real-world dataset, Snapshot Safari [16]. Previous work suggests that fog nodes may offer communication and scalability advantages over flat federated learning systems. However, a detailed analysis of these systems-level considerations is beyond the scope of this work.

FoggyTrust retains a key advantage over FLTrust in this setting: inter-group heterogeneity (e.g., across biomes) does not adversely affect trust score estimation, as trust is computed within locally homogeneous fog nodes. In contrast, FLTrust relies on a single global reference dataset, making it more susceptible to distributional mismatch across highly heterogeneous environments.

That said, the Snapshot Safari dataset introduces substantial intra-group variability, as data collected within a single biome remains inherently unstandardized. Variations in environmental conditions can induce significant local heterogeneity which include weather, illumination (e.g., day versus night), and camera positioning. These factors may degrade the reliability of trust estimation even within fog nodes.

As a result, the benefits of trust-based aggregation become less pronounced in this setting. In particular, the additional computational and statistical overhead associated with trust score estimation may outweigh its robustness advantages, with simpler methods such as FedAvg exhibiting competitive or superior performance.

VI. CONCLUSION

In this work, we introduced FoggyTrust, a hierarchical trust-based aggregation framework for federated learning in heterogeneous environments. Our results demonstrate that FoggyTrust consistently outperforms FLTrust on more challenging datasets and under adversarial settings, particularly when the assumption of a globally representative reference dataset is violated. By localizing trust computation within relatively homogeneous client groups, FoggyTrust improves the reliability of trust scores and enhances robustness against attacks that manipulate update directions.

A key takeaway from this work is that hierarchical design, despite introducing additional system complexity, can yield

substantial benefits in federated learning. In particular, decoupling local trust estimation from global aggregation enables modularity: different aggregation strategies (e.g. SCAFFOLD) can be incorporated at higher levels to address phenomena such as client drift. This highlights the potential of combining robustness mechanisms with heterogeneity-aware optimization techniques.

We further evaluated FoggyTrust on the Snapshot Safari dataset, demonstrating its applicability in real-world, highly heterogeneous settings. While results suggest that trust-based methods may incur additional overhead in such environments, FoggyTrust narrows the performance gap between robust and non-robust methods and provides a principled framework for handling structured heterogeneity. These findings indicate potential for deployment in safety-critical applications, such as distributed sensor networks or wildlife monitoring systems, where resilience to corrupted or adversarial inputs is essential.

Several directions remain for future exploration. First, a more detailed systems-level analysis of FoggyTrust is needed, particularly with respect to communication efficiency, scalability, and latency in real-world deployments. Second, improving robustness to data poisoning attacks remains an open challenge. Third, adaptive or learned grouping strategies could replace manually defined fog nodes, enabling the system to dynamically discover latent structure in client distributions. Finally, integrating more advanced second-level aggregation methods or jointly optimizing trust estimation and aggregation may further enhance performance in highly heterogeneous and adversarial environments.

VII. ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to Dr. Stephanie Gil and Karen Li for their invaluable feedback and guidance throughout this project. Their insights and support greatly contributed to the development and refinement of this work.

TABLE I
DEFAULT PARAMETERS USED FOR FL EXPERIMENTS

	Explanation	MNIST	Fashion-MNIST	CIFAR-10	Snapshot Safari
n	# clients	100			30
τ	# clients selected in each iteration	n			
R_l	# local iterations	1			
R_g	# global iterations	2000	2500	1500	200
b	batch size	32			64
$\alpha \cdot \beta$	combined learning rate	3×10^{-4}	6×10^{-3}	2×10^{-4}	1.5×10^{-4}
$\frac{m}{n}$	fraction of malicious clients (%)	20			
m	# malicious clients	20			6
f	Krum parameter	m			
k	Trim-mean parameter	m			
$ D_0 $	size of each root dataset	100			

TABLE II
THE TESTING ACCURACY SCORES OF DIFFERENT FL METHODS UNDER DIFFERENT ATTACKS AND THE ATTACK SUCCESS RATES OF THE SCALING ATTACKS FOR THE **MNIST** DATASET. THE RESULTS FOR THE SCALING ATTACKS FORMAT THE ATTACK SUCCESS RATE OF THE TARGETED ATTACK BELOW EACH TEST ACCURACY. *OUR CONTRIBUTIONS

	FedAvg	Krum	Trim-Mean	Median	FLTrust	*FoggyTrust (FedAvg)	*FoggyTrust (FedAdam)	*FoggyTrust (SCAFFOLD)
No Attack	0.9550	0.8966	0.9400	0.9385	0.9522	0.9627	0.9549	0.9626
LF Attack	0.9377	0.9007	0.8915	0.9412	0.9492	0.9480	0.9520	0.9491
Krum Attack	0.8921	0.0983	0.7979	0.1083	0.9391	0.9366	0.9362	0.9290
Trim Attack	0.7489	0.8986	0.7043	0.6780	0.9199	0.9531	0.9453	0.9479
Scaling Attack	0.0980	0.8855	0.6677	0.8788	0.9386	0.9604	0.9465	0.9580
	1.0000	0.0146	0.0340	0.0095	0.0038	0.0058	0.0068	0.0049

TABLE III
THE TESTING ACCURACY SCORES OF DIFFERENT FL METHODS UNDER DIFFERENT ATTACKS AND THE ATTACK SUCCESS RATES OF THE SCALING ATTACKS FOR THE **FASHION-MNIST** DATASET. THE RESULTS FOR THE SCALING ATTACKS FORMAT THE ATTACK SUCCESS RATE OF THE TARGETED ATTACK BELOW EACH TEST ACCURACY. *OUR CONTRIBUTIONS

	FedAvg	Krum	Trim-Mean	Median	FLTrust	*FoggyTrust (FedAvg)
No Attack	0.8894	0.8198	0.8650	0.8578	0.8898	0.8888
LF Attack	0.8943	0.8408	0.7670	0.8326	0.8813	0.8784
Krum Attack	0.1000	0.1000	0.6961	0.7121	0.8655	0.8793
Trim Attack	0.1000	0.8389	0.7218	0.7267	0.8952	0.8791
Scaling Attack	0.1000	0.8378	0.1000	0.1000	0.8865	0.8952
	1.000	1.000	1.000	1.000	0.000	0.000

TABLE IV
THE TESTING ACCURACY SCORES OF DIFFERENT FL METHODS UNDER DIFFERENT ATTACKS AND THE ATTACK SUCCESS RATES OF THE SCALING ATTACKS FOR THE **CIFAR-10** DATASET. THE RESULTS FOR THE SCALING ATTACKS FORMAT THE ATTACK SUCCESS RATE OF THE TARGETED ATTACK BELOW EACH TEST ACCURACY. *OUR CONTRIBUTIONS

	FLTrust	*FoggyTrust (FedAvg)	*FoggyTrust (FedAdam)	*FoggyTrust (SCAFFOLD)
No Attack	0.5334	0.5396	0.5384	0.5501
LF Attack	0.5281	0.5001	0.5165	0.5143
Krum Attack	0.3213	0.4837	0.4957	0.4918
Trim Attack	0.3257	0.5165	0.4671	0.5230
Scaling Attack	0.5036	0.5211	0.5165	0.5559
	0.0060	0.0371	0.0060	0.0010

REFERENCES

- [1] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, “Federated learning: Strategies for improving communication efficiency,” *CoRR*, vol. abs/1610.05492, 2016. [Online]. Available: <http://arxiv.org/abs/1610.05492>
- [2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” 2023. [Online]. Available: <https://arxiv.org/abs/1602.05629>
- [3] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “Fltrust: Byzantine-robust federated learning via trust bootstrapping,” *CoRR*, vol. abs/2012.13995, 2020. [Online]. Available: <https://arxiv.org/abs/2012.13995>
- [4] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, “Scaffold: Stochastic controlled averaging for federated learning,” 2021. [Online]. Available: <https://arxiv.org/abs/1910.06378>
- [5] Google AI Blog, “Federated learning: Collaborative machine learning without centralized training data,” <https://ai.googleblog.com/2017/04/federated-learning-collaborative.html>, 2017, accessed: 2026-04-30.
- [6] M. Mammadli, R. M. Hewett, and M. J. Harmsen, “A systematic review of federated learning applications for biomedical data,” *BioMedInformatics*, vol. 2, no. 4, pp. 745–760, 2022. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9619858/>
- [7] B. Biggio, B. Nelson, and P. Laskov, “Poisoning attacks against support vector machines,” in *Proceedings of the 29th International Conference on Machine Learning (ICML 2012)*, 2012. [Online]. Available: <https://arxiv.org/abs/1206.6389>
- [8] B. Nelson, M. Barreno, F. J. Chi, A. D. Joseph, B. I. P. Rubinstein, U. Saini, C. Sutton, J. D. Tygar, and K. Xia, “Exploiting machine learning to subvert your spam filter,” in *First USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET 08)*. San Francisco, CA: USENIX Association, Apr. 2008. [Online]. Available: <https://www.usenix.org/conference/leet-08/exploiting-machine-learning-subvert-your-spam-filter>
- [9] M. Fang, X. Cao, J. Jia, and N. Z. Gong, “Local model poisoning attacks to byzantine-robust federated learning,” in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/fang>
- [10] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, “Analyzing federated learning through an adversarial lens,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 2019, pp. 634–643. [Online]. Available: <https://proceedings.mlr.press/v97/bhagoji19a.html>
- [11] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How to backdoor federated learning,” in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, S. Chiappa and R. Calandra, Eds., vol. 108. PMLR, 2020, pp. 2938–2948. [Online]. Available: <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
- [12] C. Xie, K. Huang, P.-Y. Chen, and B. Li, “Dba: Distributed backdoor attacks against federated learning,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=rkgySOVFvr>
- [13] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [14] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*, 2018.
- [15] S. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, “Adaptive federated optimization,” 2021. [Online]. Available: <https://arxiv.org/abs/2003.00295v5>
- [16] LILA BC (Labeled Information Library of Alexandria: Biology and Conservation), “Snapshot safari 2024 expansion,” <https://lila.science/datasets/snapshot-safari-2024-expansion/>, 2024, camera trap dataset with 4M+ images across 15 projects and 151 categories.
- [17] Y. LeCun, C. Cortes, and C. Burges, “Mnist handwritten digit database,” *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, vol. 2, 2010.
- [18] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms,” 2017. [Online]. Available: <https://arxiv.org/abs/1708.07747>
- [19] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009, technical Report.
- [20] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>